Information Sciences 199 (2012) 167–178

Contents lists available at SciVerse ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Prediction of disulfide bonding pattern based on a support vector machine and multiple trajectory search

Hsuan-Hung Lin [a,b], Lin-Yu Tseng [c,d,*]

[a] Department of Applied Mathematics, National Chung Hsing University, 250, Kuo Kuang Road, Taichung 402, Taiwan, ROC
[b] Department of Management Information System, Central Taiwan University of Science and Technology, 666, Buzih Road, Beitun District, Taichung 40601, Taiwan, ROC
[c] Department of Computer Science and Communication Engineering, Providence University, 200, Chung Chi Road, Taichung 433, Taiwan, ROC
[d] Department of Computer Science and Engineering, National Chung Hsing University, 250, Kuo Kuang Road, Taichung 402, Taiwan, ROC

ARTICLE INFO

Article history:
Received 12 November 2009
Received in revised form 27 October 2010
Accepted 22 February 2012
Available online 1 March 2012

Keywords:
Disulfide bonding pattern
Support vector machine
Multiple trajectory search

ABSTRACT

To determine protein folding, accurately predicting the connectivity pattern of disulfide bridges can significantly reduce the search space, helping to solving the protein-folding problem. Therefore, developing an effective means of predicting disulfide connectivity patterns facilitates the estimation of the three-dimensional structure of a protein and its function. To our knowledge, with the prior knowledge of the bonding states of cysteines, the highest accuracy rate in the literature for predicting the overall disulfide connectivity pattern (Q_p) is 74.4% for dataset SP39. Dataset SP39 is conventionally adopted to predict disulfide connectivity. This work presents a novel classifier based on the support vector machine (SVM) that incorporates features of position-specific scoring matrix (PSSM), normalized bond lengths, the predicted secondary structure of protein, and indices for the physicochemical properties of amino acid. The support vector machine is trained to derive the connectivity probabilities of cysteine pairs. Additionally, an evolutionary algorithm called the multiple trajectory search (MTS) is integrated with the SVM model to tune the SVM parameters and window sizes for the above features. Moreover, the disulfide connectivity pattern is identified by using the maximum weight perfect matching algorithm. Experimental results indicate that the accuracy rate for predicting the overall disulfide connectivity pattern (Q_p) reaches 79.8% when tested using the same dataset SP39.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Disulfide bonds significantly impacts stabilization of protein conformations. To predict protein folding, accurately predicting of disulfide bridges can significantly reduce the search space [16,28]. Predicting the disulfide bonding pattern helps, to a certain extent, determination of the three-dimensional structure of a protein and its function because disulfide bonds impose geometrical constraints on the protein backbones. Recent works have demonstrated that the disulfide bonding patterns and protein structures are closely related to each other [9,33].

Predicting the disulfide bond requires addressing two problems. First, the disulfide bonding states must be determined, i.e. a cysteine may be in an oxidized or reduced state and one must determine whether or not a cysteine is oxidized. Second,

* Corresponding author at: Department of Computer Science and Communication Engineering, Providence University, 200, Chung Chi Road, Taichung 433, Taiwan, ROC. Tel.: +886 4 26328001x18314; fax: +886 4 26530041.
E-mail addresses: shlin@ctust.edu.tw (H.-H. Lin), lytseng@pu.edu.tw (L.-Y. Tseng).

0020-0255/$ - see front matter © 2012 Elsevier Inc. All rights reserved.
doi:10.1016/j.ins.2012.02.035

the disulfide bonding pattern must be predicted, with the goal of determining whether individual pairs of cysteines contain a disulfide bond. Significant progress has been made recently in predicting disulfide bonding states. Several methods have been developed based on statistical analysis [14], neural network [12,22], and support vector machine [8], which can accurately predict the bonding state of cysteines with accuracy rates ranging from 81% to 90%. Moreover, several methods have been devised to predict disulfide bonding patterns. Fariselli and Casadio [10] pioneered the first method by reducing disulfide connectivity to a graph matching problem in which vertices represent oxidized cysteines; in addition, edges were labeled according to the interaction strength, i.e. contact potential, in the associated pair of cysteines. The Monte Carlo simulated annealing method had been adopted to derive the optimal values of contact potentials. Finally, the disulfide bridges were derived by finding the maximum weight perfect matching. Fariselli et al. [11] improved upon their previous findings by using neural networks to estimate cysteine pairwise interactions. Vullo and Frasconi [34] constructed an ad hoc recursive neural network for scoring labeled undirected graphs that represent connectivity patterns to significantly increase the prediction accuracy rate of bonding pattern from 34% to 44%. Baldi et al. [3] and Ferrè and Clote [13] improved the prediction accuracy by using two-dimensional recursive neural networks and the diresidue neural network, respectively, to predict connectivity probabilities between cysteine pairs. Ferrè and Clote [13] also trained the predictive model based on secondary structure information and diresidue frequencies. However, Tsai et al. [31] predicted connectivity probabilities between cysteine pairs by using the support vector machine. Local sequence profiles of a protein and linear distance of cysteines in the protein were the features used to train the support vector machine. The above methods are based on reducing the connectivity pattern prediction problem to the maximum weight perfect matching problem. In contrast, the following four methods are not based on this reduction. Chen and Hwang [7] determined the bonding pattern directly by using the support vector machine. Coupling between any two cysteines located in individual local sequences (the local evolutionary information derived from the position-specific scoring matrix), the cysteine spacing patterns, and the amino acid content are the features that they used to train the support vector machine. Zhao et al. [36] assumed that two proteins with similar cysteine separation share the same disulfide connectivity pattern, so they used a simple feature called cysteine separations profiles (CSP) to predict the connectivity patterns. Chen et al. [6] developed a two-level model, in which the protein local information was used as a pair-wise feature in the first level. Inputs in the second level were the scores of the first level and the global information such as the cysteine ordering, protein length and cysteine separations profiles. Lu et al. [21] achieved a prediction accuracy of 73.9% by using a genetic algorithm to optimize the feature selection for SVM. However, Song et al. [29] obtained a prediction accuracy of 74.4% based on use of multiple sequence vectors and the secondary structure. The prediction accuracy is the best one in the literature. Rubinstein and Fiser [26] recently analyzed the correlated mutation patterns in multiple sequence alignments to predict the disulfide bond connectivity. This work, adjusts the parameters of the SVM and the window sizes of the features by using a novel evolutionary algorithm called the multiple trajectory search, and integrates it with the SVM training. The accuracy rate of the proposed algorithm achieves an accuracy rate of 79.8% when testing on the same dataset SP39.

1.1. Support vector machine

As a supervised learning method for classification [5], the support vector machine (SVM) is a highly effective means of solving problems in nonlinear classification, function estimation and density estimation. It also leads to many applications including image interpretation[23], image classification [35], intrusion detection of information security [27], pattern classification [18], data mining and other biotechnological fields [4,17,20]. In particular, applications of the SVM in medicine and biology have grown rapidly such as in the prediction of RNA-binding sites [30] and secondary structures [25] of proteins.

A set of training samples is given in the training of an SVM model. A training sample can be expressed as (X_i, y_i), with $X_i = (x_1, x_2, \ldots, x_n)$ and $y_i = 1$ if X_i is in one class and $y_i = -1$ if X_i is in another one. The training of SVM involves solving the following optimization problem:

$$\min_{\omega,b,\xi} \frac{1}{2}\omega^T\omega + C\sum_{i=1}^{n}\xi_i \quad \text{subject to } y_i(\omega^T\phi(X_i) + b) \geqslant 1 - \xi_i \quad \text{and} \quad \xi_i \geqslant 0,$$

where X_i is mapped to a higher dimensional space by the function ϕ; ξ_i denotes the allowed error; and Crepresents the penalty of the error. In practice, the kernel function can be expressed as

$$K(X_i, X_j) = \emptyset(X_i)^T\emptyset(X_j).$$

Several kernel functions are used to construct SVM models. In this work, the radial basis function (RBF) is used, which is given by $e^{-\gamma\|X_i - X_j\|^2}$ with the parameter $\gamma > 0$.

1.2. Multiple trajectory search

Devised for the single objective real-parameter optimization [32] the multiple trajectory search (MTS) uses multiple agents to search the solution space concurrently. Each agent conducts an iterated local search using one of three candidate local search methods. By selecting a local search method that best fits the landscape of a solution's neighborhood, an agent

may reach either a local optimum or the global optimum. Notably, the MTS can effectively solve the single objective real-parameter optimization problem.

The MTS is briefly described as follows. Further details can be found in [32]. The MTS initially generates M initial solutions based on the simulated orthogonal array (SOA). As a modified version of the orthogonal array (OA). The SOA tends to uniformly distribute these M initial solutions over a feasible solution space. The initial search range for local search methods is set to half the difference between the upper bound and the lower bound along each dimension. Afterwards, local search methods constrict the search range step by step.

The MTS consists of iterations of local searches until the termination condition is satisfied. During the first iteration, the MTS conducts local searches on all M initial solutions. However, in the following iterations, only a fixed number of best solutions are selected as foreground solutions and the MTS conducts local searches only on these foreground solutions. Three local search methods are devised in the MTS. The MTS initially evaluates the performance of the three local search methods and then, selects the one that performs best, i.e. the one that best fits the landscape of the neighborhood of the solution in hand, to perform the search. After conducting the search on foreground solutions, the MTS applies Local Search 1 to the current best solution that attempts to enhance the current best solution. Before completion of the iteration, a fixed number of best solutions are selected as the foreground solutions for the next iteration.

In the MTS, three local search methods are used to search different landscapes of the neighborhood of a solution. Local Search 1 searches along one dimension from the first dimension to the last one. Local Search 2 resembles Local Search 1 except that it only searches for approximately one-fourth of the dimensions. In both local search methods, the search range (SR) constricts to $0.5 \times \mathrm{SR}$ if the previous local search does not improve the current solution until it is lower than 1×10^{-4}. In Local Search 1, on the dimension concerned with the search, SR first subtracts the coordinate of the solution for this dimension to determine if the objective function value is improved. If improved, the search then considers the next dimension. If unimproved, the solutions is restored and then coordinate of the solution for this dimension is added by $0.5 \times \mathrm{SR}$, again to determine if the objective function value is improved. If improved, the search then considers the next dimension. If unimproved, the solution is restored and the search proceeds to consider the next dimension.

Local Search 3 differs from Local Searches 1 and 2. Local Search 3 considers three small movements along each dimension and determines heuristically the movement of the solution along each dimension. In Local Search 3, although the search is along each dimension from the first dimension to the last one, the objective function value is evaluated after searching for all dimensions. Moreover, the solution is moved to a new position only if the objective function is improved during this evaluation.

Highly effective at real-parameter optimization, the MTS won a competition held at the 2008 IEEE Congress on Evolutionary Computation. In this work, it is used to adjust the SVM parameters and the window sizes for various features. Since the parameters and the window sizes are real values, the MTS is highly appropriate for this application.

In addition to providing the definitions of the orthogonal array (OA) and the simulated orthogonal array (SOA), the Appendix lists the pseudo codes of the MTS and three local search methods.

2. Materials and methods

2.1. Datasets

Performance of the proposed method was compared with those of previous works [3,6,7,10,11,13,21,26,29,31,34], in which the same dataset SP39 was used. The SP39 dataset [34] was adopted from the SWISS-PROT database, release No. 39 [2], by applying the same filtering procedure used by Fariselli and Casadio [10] to include only high quality sequences with experimentally verified intra-chain disulfide bridge annotations. In the dataset, fewer than 20% of the sequences have more than five disulfide bridges. Hence, the dataset excluded all chains with more than 10 oxidized cysteines. Previous works used the dataset for fourfold cross-validation [10,11]. Two other datasets, i.e. SP43 and SP56, were also used to function as the testing datasets in order to conduct the holdout independent test. The dataset SP43 was prepared by Zhao et al. [36] by extracting from the SWISS-PROT database, release No. 43, containing 2 to 5 disulfide bonds and excluding the sequences included in SP39. Additionally, the sequences in dataset SP43 share less than 25% identity with each other. The SP56 dataset comprises new protein sequences from the SWISS-PROT database, release No. 56.3, containing 2 to 5 disulfide bonds. The sequences in SWISS-PROT database, release No.39, were excluded from SP56. Table 1 summarizes the number of protein sequences in the three datasets.

Table 1
Number of protein sequences for the three datasets classified based on the number of bonds.

Dataset	$B = 2$	$B = 3$	$B = 4$	$B = 5$	All
SP39	156	146	99	45	446
SP43	119	116	43	35	313
SP56	295	442	220	82	1039

2.2. Features

This subsection introduces the features of the proposed method, as listed below:

(1) *PSSM:* PSI-BLAST [1] was used to obtain the local sequence profiles. The output file contained four parts. The first part, i.e. the position-specific scoring matrix (PSSM), was used as one of the features.
(2) *Normalized cysteine position difference (NCPD)*: Let $\{c_1, c_2, \ldots, c_n\}$ denote the positions of oxidized cysteines in an ascending order. Assume that there is a disulfide bond between c_i and c_j. Then the normalized bond length was defined as $|c_i - c_j|/(c_n - c_1)$.
(3) *Predicted secondary structure (PSS)*: The predicted secondary structure obtained by applying Jones' prediction method PSIPRED [19] was used as another feature.
(4) *Physicochemical properties index of the amino acid (PH)*: These properties were considered and used as features by following the approach of Meiler et al. [24], which described the amino acids in terms of five parameters: graph shape index (PH1), polarizability (PH2), volume (PH3), hydrophobicity (PH4), and isoelectric point (PH5).

Before incorporated into SVM, the features were linearly scaled to $[-1, 1]$.

2.3. Methodology

The proposed cysteine connectivity prediction algorithm consists of the following steps:

(1) Run PSI-BLAST to obtain PSSM.
(2) Run PSIPRED to obtain PSS.
(3) Calculate NCPD, and obtain PH by examining the table.
(4) Encode the features for all possible cysteine bonding pairs.
(5) Use the multiple trajectory search to adjust the parameters of the SVM and the window sizes of PSSM, PSS, PH1, PH2, PH3, PH4 and PH5.
(6) Run SVM to obtain the probability of each possible cysteine bonding pair.
(7) Run the weighted perfect matching program to determine the cysteine connectivity pattern.

The first four steps simply prepare the features for each possible cysteine bonding pair. Previous works applied the evolutionary algorithms to select features in the first phase and then used the selected features to train the SVM model in the second phase [6,7,29,31]. In this work, the MTS and SVM training are tightly integrated with each other. Since the C and γ

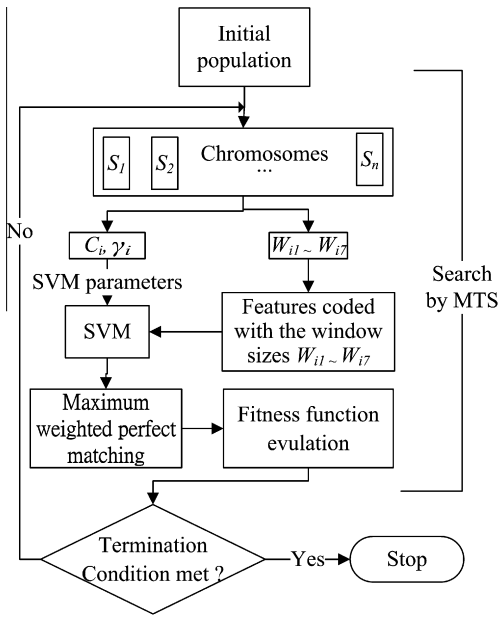


Fig. 1. Flowchart of the multiple trajectory search for adjusting the SVM parameters and the window sizes of features including PSSM, PSS, PH1, PH2, PH3, PH4 and PH5.

Table 2
Comparison of prediction accuracies between this work with those of other works.

Methods	$B = 2$		$B = 3$		$B = 4$		$B = 5$		$B = 2–5$	
	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)
MC graph-matching[a]	56	56	21	36	17	37	2	21	29	38
NN graph-matching[b]	68	68	22	37	20	37	2	26	34	42
BiRnn-2 profile[c]	73	73	41	51	24	37	13	30	44	49
2D-Rnn profile[d]	74	74	51	61	27	44	11	41	49	56
dNN2[e]	62	–	40	–	55	–	26	–	49	–
SVM[f]	74	74	61	69	30	40	12	31	55	57
SVM profile DOC_{log}[g]	79	79	53	62	55	70	58	71	63	70
2-level SVM[h]	85	–	67	–	57	–	58	–	70	–
GA + SVM[i]	85.7	85.7	74.6	79.7	63.2	77.1	47.6	71.4	73.9	79.2
SVR[j]	86.5	86.5	67.1	72.6	78.8	84.8	46.8	64.0	74.4	77.9
CMA[k]	73	73	69	71	61	64	–	59	–	–
This work	92.3	92.3	76.0	80.4	78.8	82.1	51.1	64.0	79.8	80.9

[a] Fariselli and Casadio [10].
[b] Fariselli et al. [11].
[c] Vullo and Frasconi [34].
[d] Baldi et al. [3].
[e] Ferrè and Clote [13].
[f] Chen and Hwang [7].
[g] Tsai et al. [31].
[h] Chen et al. [6].
[i] Lu et al. [21].
[j] Song et al. [29].
[k] Rubinstein and Fiser [26].

parameter values for the SVM are critical to the classification accuracy of the SVM, selecting proper values of C and γ is of priority concern. In practice, the parameter value is selected using the conventional grid search. However, this approach is extremely time-consuming. In this work, the MTS is integrated with SVM training to select not only C and γ parameter values, but also the window sizes for obtaining PSSM, PSS and PH1 to PH5 (step (5)). A chromosome is encoded as $S_i = (C_i, \gamma_i, W_{i1}, W_{i2}, W_{i3}, W_{i4}, W_{i5}, W_{i6}, W_{i7})$, where C_i and γ_i denote the log values of the parameters C and γ, and W_{i1} to W_{i7} denote the window sizes for PSSM, PSS and PH1 to PH5, respectively. The fitness function is defined as the accuracy of SVM in terms of predicting disulfide connectivity. Fig. 1 displays the flowchart of the integration of the MTS and SVM training. In this application, the MTS is used to search for an optimal set of parameters $S_o = (C_o, \gamma_o, W_{o1}, W_{o2}, W_{o3}, W_{o4}, W_{o5}, -, \gamma_o, W_{o1}, W_{o2}, W_{o3}, W_{o4}, W_{o5}, W_{o6}, W_{o7})$ within a hypercube in $\mathbb{R}^9$. Restated, N equals 9 because there are 9 parameters. The hypercube is defined by the lower bound and the upper bound for each parameter.

After the parameters are adjusted, a SVM model is derived for estimating the probability of each possible cysteine bonding pair. The SVM program used in this work is adopted from LIBSVM [5]. The features of each possible cysteine bonding pair are then input to the SVM model to obtain the probability of this pair (step (6)). After the probabilities of all possible pairs have been derived, the maximum weighted perfect matching program [15] is run to determine the cysteine connectivity pattern (step (7)).

2.4. Maximum weighted perfect matching

When a test protein is given, the connectivity probability of each cysteine pair is computed according to the trained SVM model. A complete graph is then constructed with nodes denoting all of the cysteines and the weight associated with each edge representing the disulfide connectivity probability of the pair of cysteines that are incident to this edge. The Gabow's

Table 3
Comparison of prediction accuracies of this work with those of other approaches by using SP39 dataset as the training dataset and SP43 dataset as the testing dataset.

Methods	$B = 2$	$B = 3$	$B = 4$	$B = 5$	$B = 2–5$
2-Level SVM[a]	77	59	56	31	63
SVR[b]	82	59	43	28	62
This work	83.2	62.1	76.7	48.6	70.6

[a] Chen et al. [6].
[b] Song et al. [29].

Table 4
Prediction accuracy of the proposed method by using SP39 dataset as the training dataset and SP56 datasets as the testing dataset.

Dataset	$B = 2$		B B = 3		$B = 4$		$B = 5$		$B = 2–5$	
	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)	Q_P(%)	Q_C(%)
SP56	76.6	76.9	71.7	77.8	67.7	76.9	40.2	59.5	69.8	75.0

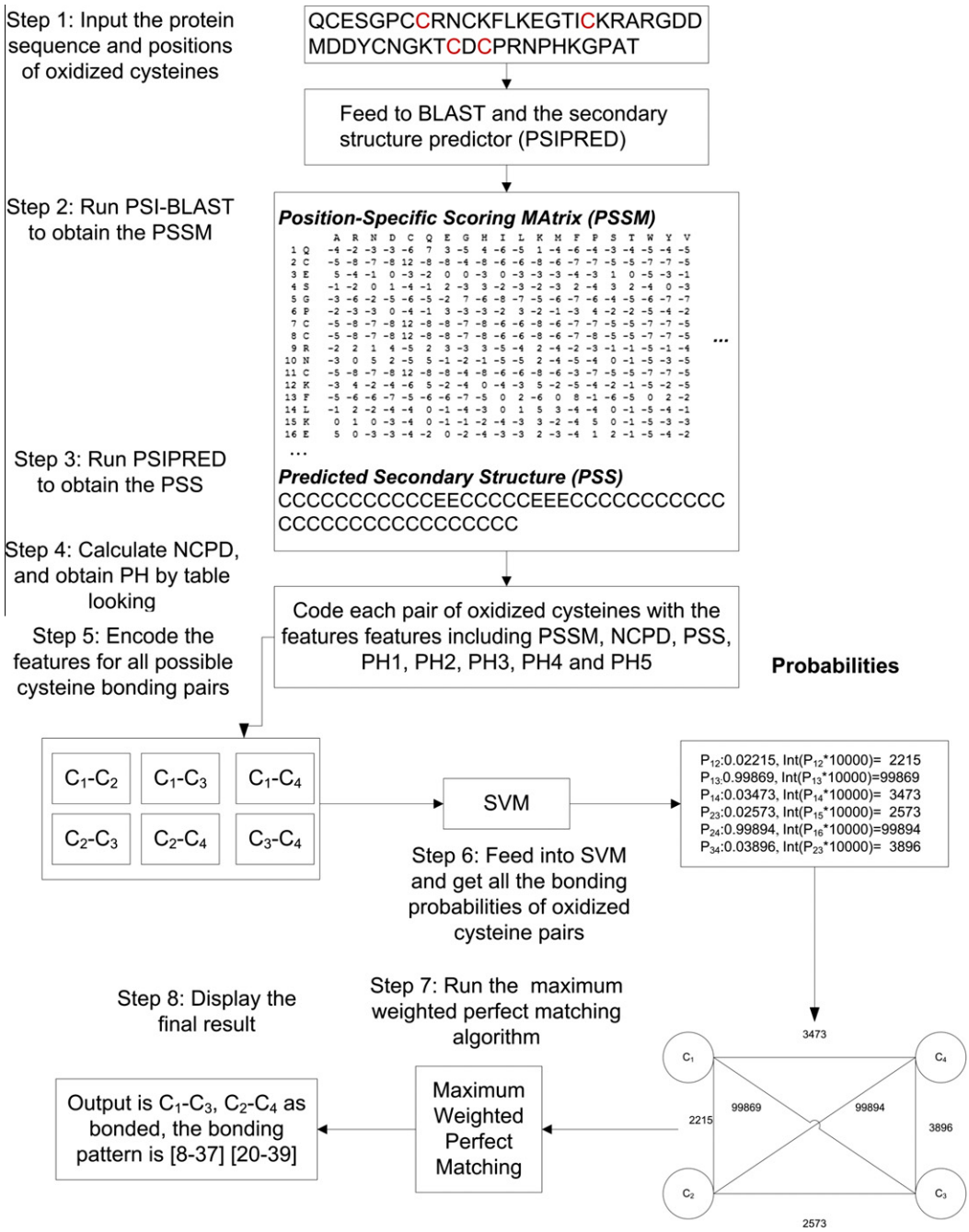
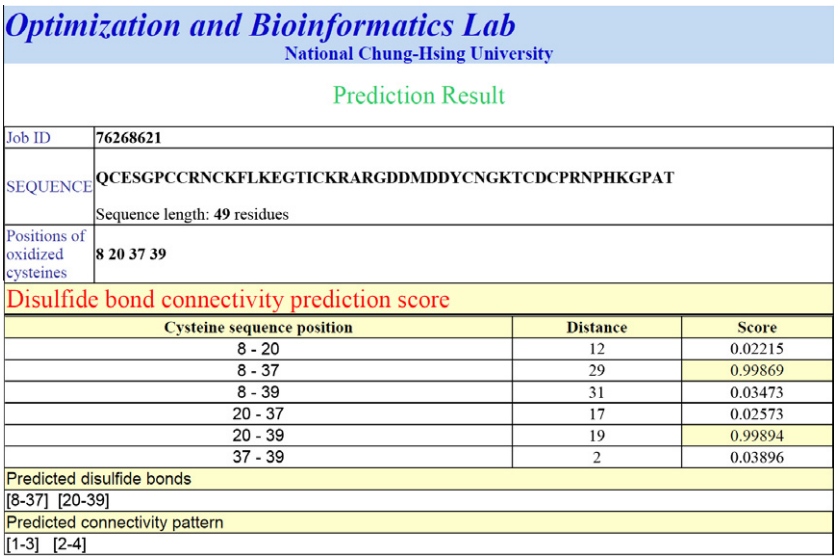


Fig. 2. Example of the prediction flow for the web service.



Optimization and Bioinformatics Lab
National Chung-Hsing University

Prediction Result

Job ID	76268621
SEQUENCE	QCESGPCCRNCKFLKEGTICKRARGDDMDDYCNGKTCDCPRNPHKGPAT Sequence length: 49 residues
Positions of oxidized cysteines	8 20 37 39

Disulfide bond connectivity prediction score

Cysteine sequence position	Distance	Score
8 - 20	12	0.02215
8 - 37	29	0.99869
8 - 39	31	0.03473
20 - 37	17	0.02573
20 - 39	19	0.99894
37 - 39	2	0.03896

Predicted disulfide bonds
[8-37] [20-39]
Predicted connectivity pattern
[1-3] [2-4]

Fig. 3. Output screen of the web service for illustrative purposes.

algorithm [15] is then applied to obtain the maximum weight perfect matching. Because the Gabow's algorithm can only be applied with edges labeled in integer, the disulfide connectivity probability of a pair of cysteines is multiplied by 10000 and then truncated into an integer to represent the weight associated with the edge. This matching represents the prediction outcome of the disulfide connectivity pattern.

3. Results and discussion

Performance of the prediction algorithm is evaluated using two accuracy indices Q_P and Q_C:

$$Q_P = C_P/T_P \quad \text{and} \quad Q_C = C_C/T_C,$$

where C_P represents the number of proteins whose bonding patterns are correctly predicted; T_P represents the total number of proteins in the test set; C_C represents the number of disulfide bridges that are correctly predicted; and T_C represents the total number of disulfide bridges in test proteins.

Table 2 summarizes the accuracy rate of the proposed method in comparison with those of other approaches. All methods were tested on the SP39 dataset. According to Table 2, prediction accuracy of the proposed method is superior to all other methods in previous works [3,6,7,10,11,13,21,26,29,31,34].

3.1. Holdout test on new sequences from SP43 and SP56

Accuracy of the proposed method in predicting disulfide connectivity for new protein sequences is further verified by performing a more rigorous holdout test adopted previously by Zhao et al. [36] and Chen et al. [6], in which the SP39 dataset is used as the training dataset and the SP43 dataset is used as the testing dataset. Table 3 summarizes those results. Another experiment used dataset SP39 as the training dataset and SP56 dataset as the testing dataset. Table 4 summarizes those results. Tables 3 and 4, reveal that the proposed method achieves an accuracy rate Q_P of approximately 70% for both SP43 and SP56 datasets. This finding demonstrates the reliability of the proposed method.

A higher prediction accuracy of the proposed algorithm may be owing to selection of the proper features and the applicability of the MTS for adjusting the SVM parameter values and the window sizes of some features.

3.2. Online web service program

This work also implements an on-line service program that can be accessed online at http://120.107.8.16/disulfide or http://140.120.14.136/disulfide. The program allows users to submit a protein sequence with the positions of oxidized cysteines, and predicts the disulfide connectivity pattern. The server either displays the prediction results on web pages or sends the results to the user by e-mail.

Fig. 2 illustrates an example of the prediction flow of the web service. The input protein is named Disintegrin echistatin-alpha. This protein contains 6 cysteines; the protein contains two disulfide bonds, i.e. Cys8-Cys37 and Cys20-Cys39. Fig. 2

illustrates the flow of the web service that accurately predicts the disulfide connectivity pattern. Fig. 3 displays the output screen of the web service for illustrative purposes.

4. Conclusions

Given the importance of feature selection and/or extraction in the classification of patterns, this study, predicted cysteine connectivity patterns based on four features. Experimental results indicate that the proposed method achieves an accuracy of 79.8% tested using fourfold cross-validation with SP39 dataset, which is better than the prediction performance reported in previous studies. Our results also indicate that the proposed method achieves prediction accuracies as high as 70.6% and 69.8% when the SP39 dataset is used as the training dataset, and the SP43 and SP56 datasets as the testing datasets, respectively.

In conclusion, based on the use of the SVM with the MTS to adjust the optimal parameters of SVM and the window sizes for various features, the proposed disulfide connectivity pattern prediction method, can accurately predict a disulfide connectivity pattern efficiently.

Acknowledgements

The authors thank National Science Council of the Republic of China, Taiwan (Contract No. NSC98-2221-E005-049-MY3) and Central Taiwan University of Science and Technology (Contract No. CTU99-P-33) for partially supporting this research work. The anonymous reviewers are commended for their valuable comments.

Appendix A. Orthogonal array and simulated orthogonal array

Orthogonal arrays used in the experimental design methods are described briefly. Assume that in an experiment, there are k factors and each factor has q levels. Importantly, q^k experiments must be performed to identify the optimum setting of each factor's level. Additionally, testing all q^k combinations is often impossible or cost ineffective. A small but representative sample of combinations must be sampled for testing. For this purpose, orthogonal arrays were developed. In an experiment with k factors and each factor has q levels, an orthogonal array $OA(n, k, q, t)$ is an array with n rows and k columns. This is a representative sample of n testing experiments that satisfies the following three conditions: (1) For the factor in any column, the same number of times occurs for each level; (2) for the t factors in any t columns, each combination of q levels occurs the same number of times; and (3) the selected combinations are uniformly distributed over the entire space of all possible combinations. In the notation $OA(n, k, q, t)$, n denotes the number of experiments; k denotes the number of factors; q denotes the number of levels of each factor; and t denotes the strength. Another often used notation for the orthogonal array is $L_n(q^k)$. In this notation, t is omitted and is always set to 2. Table A1 presents an $L_8(2^7)$ orthogonal array as an illustrative example.

Orthogonal arrays exist for only some specific n's and k's, explaining why using OA in some applications is infeasible. This work proposes the simulated OA (SOA). Despite satisfying only the first of the above three conditions, SOA of nearly any size can be easily constructed.

Assume there are k factors and each factor has q levels. Additionally, an $m \times k$ simulated orthogonal array $SOA_{m\times k}$ with m as a multiple of q can be generated as follows.

For each column of $SOA_{m\times k}$, a random permutation of $0, 1, \ldots, q-1$ is generated and denoted as sequence C. The elements in C are then selected sequentially and filled in a randomly selected empty entry of the column. If all elements in C are selected, the process selects elements again from the beginning of C. Therefore, in each column of $SOA_{m\times k}$, each q element appears the same number of times (condition 1).

Table A1
$L_8(2^7)$ orthogonal array.

Test No.	Factors							Evaluation value (E_i)
	1	2	3	4	5	6	7	
1	0	0	0	0	0	0	0	E_1
2	0	0	0	1	1	1	1	E_2
3	0	1	1	0	0	1	1	E_3
4	0	1	1	1	1	0	0	E_4
5	1	0	1	0	1	0	1	E_5
6	1	0	1	1	0	1	0	E_6
7	1	1	0	0	1	1	0	E_7
8	1	1	0	1	0	0	1	E_8

Appendix B. Pseudo codes for MTS and three local search methods

```
Multiple Trajectory Search
/*Generate Minitial solutions*/
Build simulated orthogonal array SOA_{M×N}
For i = 1 to M
  For j = 1 to N
    X_i[j] = l_i + (u_i − l_i) * SOA[i, j]/(M − 1)
  End For
End For
Evaluate function values of X_i's
For i = 1 to M
  Enable[i] ← TRUE
  Improve[i] ← TRUE
  SearchRangeX_i = (UPPER_BOUND − LOWER_BOUND)/2
End For
While (#ofEvaluation ⩽ predefined_max_evaluation)
  Forі = 1 to M
    If Enable[i]=TRUE
    ThenGradeX_i ← 0
      LS1_TestGrade ← 0
      LS2_TestGrade ← 0
      LS3_TestGrade ← 0
      For j = 1 to #ofLocalSearchTest
        LS1_TestGrade ← LS1_TestGrade + LocalSearch1(X_i, SearchRangeX_i)
        LS2_TestGrade ← LS2_TestGrade + LocalSearch2(X_i, SearchRangeX_i)
        LS3_TestGrade ← LS3_TestGrade + LocalSearch3(X_i, SearchRangeX_i)
      End For
      Choose the one with the best TestGrade and
      let it be LocalSearchK    /* K may be 1, 2, or 3 */
      For j = 1 to #ofLocalSearch
        GradeX_i ← GradeX_i + LocalSearchK (X_i, SearchRangeX_i)
      End For
    End If
  End For
  For I = 1 to #ofLocalSearchBest
    LocalSearch1(BestSolution, SearchRangeBestSolution)
  End For
  For i = 1 to M
    Enable[i] ← FALSE
  End For
  Choose #of Foreground X_i's whose GradeX_i are best among the Msolutions and set their corresponding Enable[i] to
  TRUE
End While

Function LocalSearch1(X_k, SR)
If Improve[k] = FALSE
Then SR = SR/2
    If SR < 1e−4
    Then SR ← (UPPER_BOUND-LOWER_BOUND) * 0.4
  End If
End If
Improve[k] ← FALSE
For i = 1 to N
  X_k[i] ← X_k[i] − SR
    If X_k is better than current best solution
      Then grade ← grade + BONUS1
        Update current best solution
    End If
```

(continued on next page)

```
    If function value of X_k is the same
      Then restore X_k to its original value
    Else
      If function value of X_k degenerates
        Then restore X_k to its original value
        X_k[i]_X_k[i] + 0.5 * SR
        If X_k is better than current best solution
          Then grade ← grade + BONUS1
            Update current best solution
        End If
        If function value of X_k has not been improved
          Then restore X_k to its original value
        Else
          grade ← grade + BONUS2
          Improve[k] ← TRUE
        End If
      Else
        grade← grade + BONUS2
        Improve[k] ← TRUE
      End If
    End If
  End For
  return grade

Function LocalSearch2(X_k, SR)
If Improve[k]=FALSE
  Then SR = SR/2
    If SR < 1e-4
      Then SR← (UPPER_BOUND-LOWER_BOUND) * 0.4
    End If
  End If
  Improve[k] ← FALSE
  For l = 1 to N
    For i = 1 to N
      r[i] ← Random{0,1,2,3}
      D[i] ← Random{-1,1}
    End For
      For i = 1 to N
      If r[i] = 0
        Then X_k[i] ←  X_k [i]− − SR * D[i]
      End If
    End For
    If X_k is better than current best solution
      Then grade ←  grade + BONUS1
        Update current best solution
  End If
  If function value of X_k is the same
    Then restore X_k to its original value
  Else
    If function value of X_k degenerates
      Then restore X_k to its original value
        For i = 0 to N
          If r[i] = 0
            Then X_k[i]_X_k[i]+0.5 * SR * D[i]
          End If
        End For
        If X_k is better than current best solution
          Then grade ←  grade + BONUS1
            Update current best solution
```

```
            End If
            If function value of X_k has not been improved
              Then restore X_k to its original value
            Else
              grade ← grade + BONUS2
              Improve[k] ← TRUE
          End If
      Else
        grade ← grade + BONUS2
        Improve[k] ← TRUE
      End If
    End If
  End For
  return grade

Function LocalSearch3(X, SR)
For i = 1 to N
  X_1  ← X's ith coordinate is increased by 0.1
  Y_1  ← X's ith coordinate is decreased by 0.1
  X_2  ← X's ith coordinate is increased by 0.2
  If X_1 is better than current best solution
    Then grade ← grade + BONUS1
      Update current best solution
  End If
  If Y_1 is better than current best solution
    Then grade_grade+BONUS1
      Update current best solution
  End If
  If X_2 is better than current best solution
    Then grade ← grade + BONUS1
      Update current best solution
  End If
  D_1 = F(X) − F(X_1)
  If D_1  > 0//X_1 is better than X
    Then grade ← grade + BONUS2
  End If
  D_2 = F(X) − F(Y_1)
  If D_2  > 0//Y_1 is better than X
    Then grade← grade + BONUS2
  End If
  D_3 = F(X) − F(X_2)
  If D_3 > 0 //X_2 is better than X
    Then grade← grade + BONUS2
  End If
  a ← Random[0.4, 0.5]
  b ← Random[0.1, 0.3]
  c ← Random[0, 1]
  X[i] = X[i] + a(D_1 − D_2) + b (D_3 − 2D_1) + c
End For
If function value of X has not been improved
  Then restore X to its original value
Else
  grade ← grade + BONUS2
End If
return grade
```

References

[1] S.F. Altschul, T.L. Madden, A.A. Schäffer, J. Zhang, Z. Zhang, W. Miller, D.J. Lipman, Gapped BLAST and PSI-BLAST: a new generation of protein database search programs, Nucleic Acids Research 25 (17) (1997) 3389–3402.

[2] A. Bairoch, R. Apweiler, The Swiss-Prot protein sequence database and its supplement TrEMBL in 2000, Nucleic Acids Research 28 (1) (2000) 45–48.

[3] P. Baldi, J. Cheng, A. Vullo, Large-scale prediction of disulphide bond connectivity, in: L.K. Saul, Y. Weiss, L. Bottou (Eds.), Advances in Neural Information Processing Systems, vol. 17, MIT Press, Cambridge, MA, 2005, pp. 97–104.
[4] M.P.S. Brown, W.N. Grundy, D. Lin, N. Cristianini, C. Sugnet, T.S. Furey, J.M. Ares, D. Haussler, Knowledge-based analysis of microarray gene expression data using support vector machines, PNAS 97 (1) (2000) 262–267.
[5] C.C. Chang, C.J. Lin, LIBSVM: A Library for Support Vector Machines, 2001. <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.
[6] B.J. Chen, C.H. Tsai, C.H. Chan, C.Y. Kao, Disulfide connectivity prediction with 70% accuracy using two-level models, Proteins 64 (1) (2006) 246–252.
[7] Y.C. Chen, J.K. Hwang, Prediction of disulfide connectivity from protein sequences, Proteins 61 (3) (2005) 507–512.
[8] Y.C. Chen, Y.S. Lin, C.J. Lin, J.K. Hwang, Prediction of the bonding states of cysteines using the support vector machines based on multiple feature vectors and cysteine state sequences, Proteins 55 (4) (2004) 1036–1042.
[9] C.C. Chuang, C.Y. Chen, J.M. Yang, P.C. Lyu, J.K. Hwang, Relationship between protein structures and disulfide-bonding patterns, Proteins 53 (1) (2003) 1–5.
[10] P. Fariselli, R. Casadio, Prediction of disulfide connectivity in proteins, Bioinformatics 17 (10) (2001) 957–964.
[11] P. Fariselli, P.L. Martelli, R. Casadio, A neural network base method for predicting the disulfide connectivity in proteins, in: E. Damiani, R.J. Howlett, L.C. Jain, N. Ichalkaranje (Eds.), Knowledge based Intelligent Information Engineering Systems and Allied Technologies, IOS Press, Amsterdam, 2002, pp. 464–468.
[12] P. Fariselli, P. Riccobelli, R. Casadio, Role of evolutionary information in predicting the disulfide-bonding state of cysteine in proteins, Proteins 36 (3) (1999) 340–346.
[13] F. Ferrè, P. Clote, Disulfide connectivity prediction using secondary structure information and diresidue frequencies, Bioinformatics 21 (10) (2005) 2336–2346.
[14] A. Fiser, I. Simon, Predicting the oxidation state of cysteines by multiple sequence alignment, Bioinformatics 16 (3) (2000) 251–256.
[15] H.N. Gabow, Implementation of Algorithms for Maximum Matching on Nonbipartite Graphs, Phd Thesis, Stanford University, CA, 1973.
[16] E.S. Huang, R. Samudrala, J.W. Ponder, Ab initio fold prediction of small helical proteins using distance geometry and knowledge-based scoring functions, Journal of Molecular Biology 290 (1999) 267–281.
[17] S. Idicula-Thomas, A.J. Kulkarni, B.D. Kulkarni, V.K. Jayaraman, P.V. Balaji, A support vector machine-based method for predicting the propensity of a protein to be soluble or to form inclusion body on overexpression in *Escherichia coli*, Bioinformatics 22 (3) (2006) 278–284.
[18] B. Jin, Y.C. Tang, Y.Q. Zhang, Support vector machines with genetic fuzzy feature transformation for biomedical data classification, Information Sciences 177 (2) (2007) 476–489.
[19] D.T. Jones, Protein secondary structure prediction based on position-specific scoring matrices, Journal of Molecular Biology 292 (1999) 195–202.
[20] P.M. Kasson, J.B. Huppa, M.M. Davis, A.T. Brunger, A hybrid machine-learning approach for segmentation of protein localization data, Bioinformatics 21 (19) (2005) 3778–3786.
[21] C.H. Lu, Y.C. Chen, C.S. Yu, J.K. Hwang, Predicting disulfide connectivity patterns, Proteins 67 (2) (2007) 262–270.
[22] P.L. Martelli, P. Fariselli, L. Malaguti, R. Casadio, Prediction of the disulfide-bonding state of cysteines in proteins with hidden neural networks, Protein Engineering 15 (12) (2002) 951–953.
[23] M.E. Mavroforakis, H.V. Georgiou, N. Dimitropoulos, D. Cavouras, S. Theodoridis, Mammographic masses characterization based on localized texture and dataset fractal analysis using linear, neural and support vector machine classifiers, Artificial Intelligence in Medicine 37 (2) (2006) 145–162.
[24] J. Meiler, M. Muller, A. Zeidler, F. Schmaschke, Generation and evaluation of dimension-reduced amino acid parameter representations by artificial neural networks, Journal of Molecular Modeling 7 (9) (2001) 360–369.
[25] Y. Qi, F. Lin, K.K. Wong, A networked multi-class SVM for protein secondary structure prediction, in: WSEAS Int. Conf. on Mathematical Biology and Ecology MABE '05, Udine, Italy, 2005, pp.70–81.
[26] R. Rubinstein, A. Fiser, Predicting disulfide bond connectivity in proteins by correlated mutations analysis, Bioinformatics 24 (4) (2008) 498–504.
[27] T. Shon, J. Moon, A hybrid machine learning approach to network anomaly detection, Information Sciences 177 (18) (2007) 3799–3821.
[28] J. Skolnick, A. Kolinski, A.R. Ortiz, MONSSTER: a method for folding globular proteins with a small number of distance restraints, Journal of Molecular Biology 265 (1997) 217–241.
[29] J. Song, Z. Yuan, H. Tan, T. Huber, K. Burrage, Predicting disulfide connectivity from protein sequence using multiple sequence feature vectors and secondary structure, Bioinformatics 23 (23) (2007) 3147–3154.
[30] J. Tong, P. Jiang, Z. Lu, RISP: a web-based server for prediction of RNA-binding sites in proteins, Computer Methods and Programs in Biomedicine 90 (2) (2008) 148–153.
[31] C.H. Tsai, B.J. Chen, C.H. Chan, H.L. Liu, C.Y. Kao, Improving disulfide connectivity prediction with sequential distance between oxidized cysteines, Bioinformatics 21 (24) (2005) 4416–4419.
[32] L.Y. Tseng, C. Chen, Multiple trajectory search for large scale global optimization, in: Proceedings of 2008 IEEE Congress on Evolutionary Computation, CEC'08, Crystal City, Washington, DC, 2008, pp. 3052–3059.
[33] H.W.T. van Vlijmen, A. Gupta, L.S. Narasimhan, J. Singh, A novel database of disulfide patterns and its application to the discovery of distantly related homologs, JJournal of Molecular Biology 335 (2004) 1083–1092.
[34] A. Vullo, P. Frasconi, Disulfide connectivity prediction using recursive neural networks and evolutionary information, Bioinformatics 20 (5) (2004) 653–659.
[35] W.T. Wong, F.Y. Shih, J. Liu, Shape-based image retrieval using support vector machines, Fourier descriptors and self-organizing maps, Information Sciences 177 (8) (2007) 1878–1891.
[36] E. Zhao, H.L. Liu, C.H. Tsai, H.K. Tsai, C.H. Chan, C.Y. Kao, Cysteine separations profiles on protein sequences infer disulfide connectivity, Bioinformatics 21 (8) (2005) 1415–1420.